ESAME DI FONDAMENTI DI INFORMATICA T-2 del 9/7/2025

Proff. Enrico Denti – Ambra Molesini

Tempo a disposizione: 3h30

NOME PROGETTO ECLIPSE: CognomeNome-matricola (es. RossiMario-0000123456)
NOME CARTELLA PROGETTO: CognomeNome-matricola (es. RossiMario-0000123456)
NOME ZIP DA CONSEGNARE: CognomeNome-matricola.zip (es. RossiMario-0000123456.zip)
NOME JAR DA CONSEGNARE: CognomeNome-matricola.jar (es. RossiMario-0000123456.jar)

Si devono consegnare DUE FILE: *l'intero progetto Eclipse* e *il JAR eseguibile*

Si ricorda che compiti *non compilabili*, o che *non passino almeno 2/3 dei test* o siano *palesemente lontani da 18/30* NON SARANNO CORRETTI e causeranno la verbalizzazione del giudizio "RESPINTO".

L'Associazione Frutticoltori di Dentinia ha richiesto lo sviluppo di un'applicazione che effettui statistiche sull'andamento dei prezzi dei vari tipi di frutta, consentendone un'agile consultazione anche tramite grafici.

DOMINIO DEL PROBLEMA: un tipo di ***frutta*** è un nome, solitamente di una singola parola, di un certo frutto; per ogni tipo di frutta esistono diversi ***sottotipi,*** ognuna delle quali caratterizzata dal proprio nome (spesso anch'esso di una singola parola). La *coppia tipo di frutta/sottotipo* costituisce quindi un *identificativo univoco* dello specifico frutto.

L'Associazione Frutticoltori effettua, con cadenza circa settimanale, la ***rilevazione*** del prezzo per i tipi di frutta disponibili in stagione: ogni rilevazione è costituita dalla *coppia (data, prezzo)*. Naturalmente, i prezzi sono campionati solo nei periodi in cui quei tipi di frutta/sottotipo sono effettivamente in vendita.

OBIETTIVO: L'applicazione deve permettere all'utente di:

- innanzitutto, selezionare il tipo di frutta (es. albicocche, pesche, mele...)
- poi, selezionare il sottotipo di frutta fra quelli disponibili per lo specifico tipo di frutta

In risposta, l'applicazione deve:

- mostrare a video il prezzo minimo, massimo e medio di quello specifico sottotipo di frutta
- mostrare un istogramma con l'andamento dei prezzi nelle diverse date, per quello specifico sottotipo di frutta.

Il grafico deve permettere di mostrare anche più serie: ogni volta che l'utente ne seleziona un altro dalle combo, i dati di quel tipo di frutta/sottotipo devono essere aggiunti al grafico (e alla relativa legenda).

Un pulsantino RESET deve permettere in ogni momento di ripulire il grafico, cancellando le serie fin qui mostrate.

TEMPO STIMATO PER SVOLGERE L'INTERO COMPITO: 2h15 – 3h

PARTE 1 – Modello dei dati:	Punti 10	[TEMPO STIMATO: 45-60 minuti]
PARTE 2 – Persistenza:	Punti 10	[TEMPO STIMATO: 45-60 minuti]
PARTE 3 – Grafica:	Punti 10	[TEMPO STIMATO: 45-60 minuti]

NUMERO MINIMO DI TEST CON SUCCESSO PERCHÉ IL COMPITO SIA CORRETTO
40 su 58

JAVAFX – Parametri run configuration nei LAB

```
--module-path "C:\applicativi\moduli\javafx-sdk-23.0.2\lib"
--add-modules javafx.controls
```

Cose da ricordare

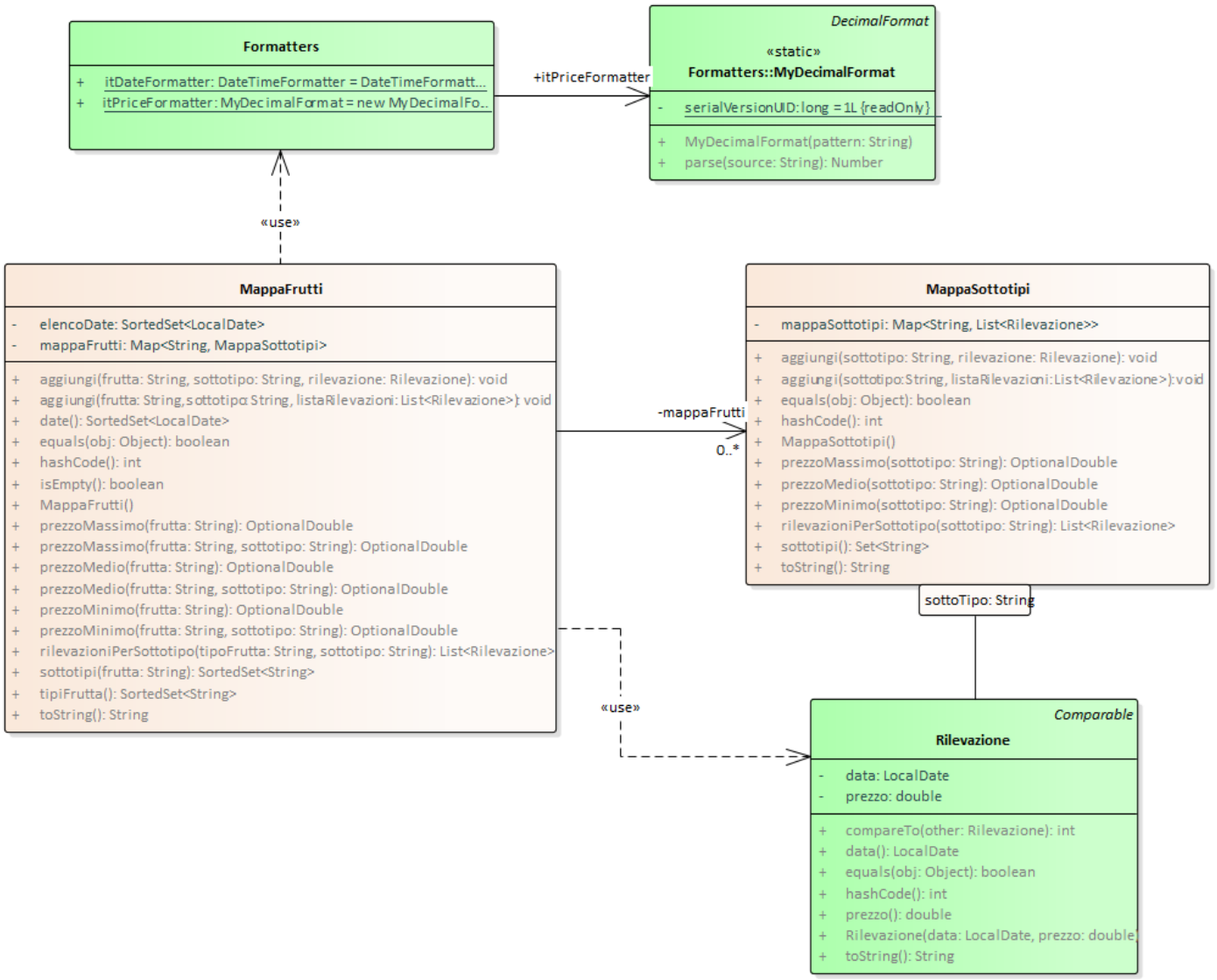
- salva costantemente il tuo lavoro: l'informatica a volte può essere "subdolamente ostile"..
- in particolare: se ora compila e stai per fare modifiche, salva la versione attuale (non si sa mai)

Checklist di consegna

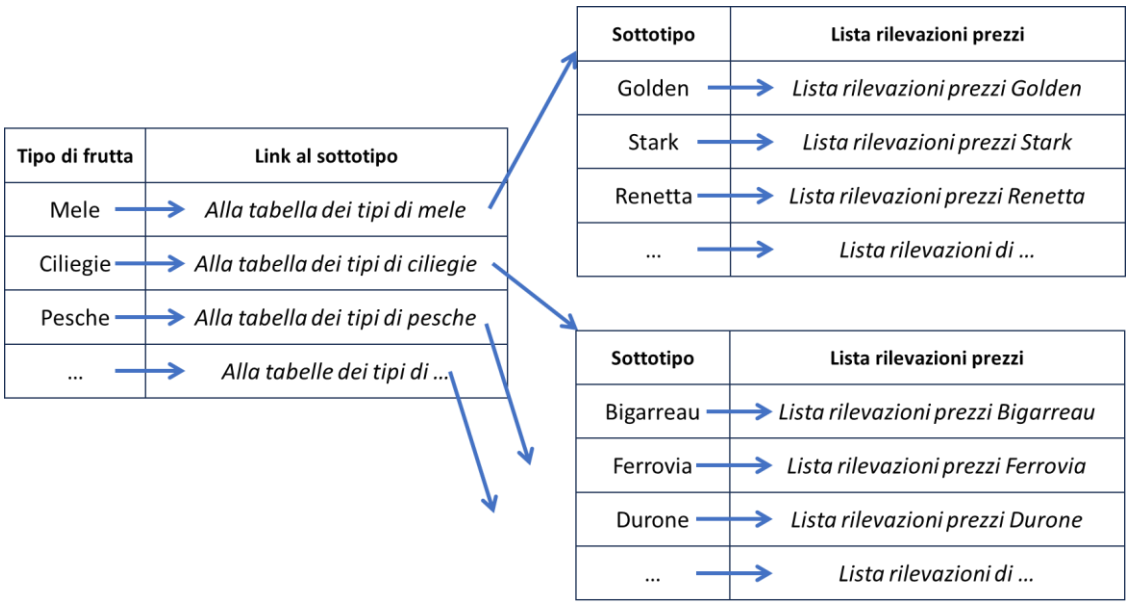
- Hai fatto un **JAR eseguibile**, che contenga cioè l'indicazione del main?
- Hai controllato che **si compili** e **ci sia tutto**? [NB: non includere il PDF del testo]
- Hai **rinominato** IL PROGETTO, lo ZIP e il JAR esattamente come richiesto?
- Hai **chiamato** la cartella del progetto esattamente come richiesto?
- Hai fatto un **unico file ZIP (NON .7z, rar o altri formati)** contenente l'intero progetto? In particolare, ti sei assicurato di aver incluso tutti i file .java (e non solo i .class)?
- **Hai consegnato DUE file distinti, ossia lo ZIP col progetto e il JAR eseguibile?**
- Su EOL, hai **premuto** il tasto "CONFERMA" per inviare il tuo elaborato?

Parte 1 – Modello dei dati (punti: 10)

Package: frutta.model **[TEMPO STIMATO: 45-60 minuti]**



La struttura dati è organizzata secondo due livelli di tabelle: il primo distingue i tipi di frutta, il secondo il sottotipo. A ogni sottotipo è associata la corrispondente lista di rilevazioni prezzi, come mostrato in figura:



SEMANTICA:

- La classe ***Formatters*** (fornita) fornisce due formattatori, rispettivamente per date in formato italiano short (standard) e per prezzi nel formato italiano con simbolo di valuta davanti al numero. Quest'ultimo lancia ***NumberFormatException*** con apposito messaggio d'errore nel caso in cui qualcosa vada storto, mentre restituisce normalmente la costante ***Double.NaN*** nel caso in cui il prezzo sia espresso dalla stringa "NaN".
- La classe ***Rilevazione*** (fornita) definisce la coppia (data, prezzo), recuperabili tramite appositi accessor. Il costruttore effettua i soliti controlli, lanciando in particolare ***IllegalArgumentException*** con apposito messaggio d'errore nel caso di prezzi negativi o infiniti. La classe è anche ***Comparable*** per data (NON per prezzo!) Sono definite *equals*, *hashCode* coerenti con tale approccio, nonché un'opportuna *toString.*
- La classe ***MappaSottotipi*** (da completare) rappresenta la mappa che associa un sottotipo di frutta alla lista di rilevazioni. Il costruttore la produce vuota: gli elementi si aggiungono poi tramite il metodo *aggiungi*.

  Il metodo *aggiungi* è definito in due varianti, con diversa lista di argomenti:
  - una prima variante (da realizzare) accetta come argomenti un sottotipo (stringa) e una ***Rilevazione***: il metodo aggiunge la rilevazione alla lista del corrispondente sottotipo. Effettua preventivamente la validazione degli argomenti di ingresso secondo le usuali linee guida, lanciando ***NullPointerException*** o ***IllegalArgumentException*** con idonei e specifici messaggi d'errore secondo necessità.
  - una seconda variante (fornita) permette, appoggiandosi alla precedente, di aggiungere una ***lista di rilevazioni*** in un'unica operazione.

  Oltre all'ovvio accessor *sottotipi*, che restituisce l'insieme dei sottotipi presenti nella mappa, sono forniti una sintetica *toString* e i seguenti metodi:
  - *rilevazioniPerSottotipo* restituisce la lista delle rilevazioni di un sottotipo; lancia ***NoSuchElement-Exception*** con adeguato messaggio d'errore nel caso la categoria richiesta sia inesistente
  - *prezzoMinimo, prezzoMedio, prezzoMassimo* (da realizzare) restituiscono, sotto forma di ***OptionalDouble***, il prezzo minimo/medio/massimo di un dato sottotipo di frutta.

- La classe ***MappaFrutti*** (da completare) rappresenta la mappa che associa un tipo di frutta alla corrispondente mappa di sottotipi. Anche questo oggetto viene prodotto vuoto dal costruttore: contiene internamente il campo *elencoDate*, che dovrà essere aggiornato dal metodo *aggiungi*.

  Anche in questo caso il metodo *aggiungi* è definito in due varianti, con diversa lista di argomenti:
  - una prima variante (da realizzare) accetta tre argomenti: il tipo di frutta (stringa), il sottotipo (stringa) e una ***Rilevazione***: dopo la validazione degli argomenti secondo le usuali linee guida, che nel caso lanciano le usuali eccezioni con adeguato messaggio d'errore, il metodo aggiunge la rilevazione alla mappa del corrispondente sottotipo.
  - una seconda variante (fornita) permette, appoggiandosi alla precedente, di aggiungere una ***lista di rilevazioni*** in un'unica operazione.

  I tre accessor *tipiFrutta*, *sottotipi* e *date* (da realizzare) devono restituire insiemi ordinati secondo l'ordine naturale.

  Sono inoltre disponibili ovvie implementazioni di *equals*, *hashCode* e *toString*, nonché i seguenti metodi (NB: per il dettaglio di quali eccezioni lanciare nei casi specifici, fare riferimento ai test forniti):
  - *rilevazioniPerSottotipo* restituisce la lista delle rilevazioni di un dato tipo e sottotipo di frutta; lancia ***NoSuchElementException*** con adeguato messaggio nel caso la categoria richiesta sia inesistente
  - *prezzoMinimo, prezzoMedio, prezzoMassimo* restituiscono, nella versione a due argomenti, il prezzo minimo/medio/massimo di un dato tipo e sottotipo di frutta, sotto forma di ***OptionalDouble***; se necessario, vengono lanciate secondo i casi ***NullPointer-Exception, IllegalArgumentException*** *o* ***NoSuchElementException*** con adeguato messaggio d'errore.

- *prezzoMinimo, prezzoMedio, prezzoMassimo* restituiscono, nella versione a un solo argomento, il prezzo minimo/medio/massimo di un dato tipo di frutta, considerando simultaneamente tutti i sottotipi, anch'esso sotto forma di ***OptionalDouble***. Anche in questi casi vengono lanciate secondo i casi ***NullPointer-Exception***, ***IllegalArgumentException*** o ***NoSuchElementException*** con adeguato messaggio d'errore.
- *isEmpty* restituisce l'esito della verifica sul contenuto attuale della mappa.

Parte 2 – Persistenza (punti: 10)

Package: frutta.persistence **[TEMPO STIMATO: 45-60 minuti]**

Il file di testo frutta.txt elenca le rilevazioni dei vari tipi e sottotipi di frutta, elencati per righe. La prima riga funge da intestazione e contiene, dopo una parola chiave iniziale, le date delle rilevazioni; tutti gli elementi sono separati dal carattere "punto e virgola". Più precisamente, la prima riga contiene, nell'ordine:

- la parola "Prodotti", scritta utilizzando indifferentemente maiuscole e minuscole
- una serie di date in formato italiano SHORT

Le righe successive dettagliano ciascuna i prezzi rilevati per uno specifico tipo e sottotipo di frutta (non esistono quindi due righe con lo stesso identificativo univoco) nelle date sopra citate, e sono anch'esse costituite da una serie di elementi separati dal carattere "punto e virgola". Più precisamente, nell'ordine sono elencati:

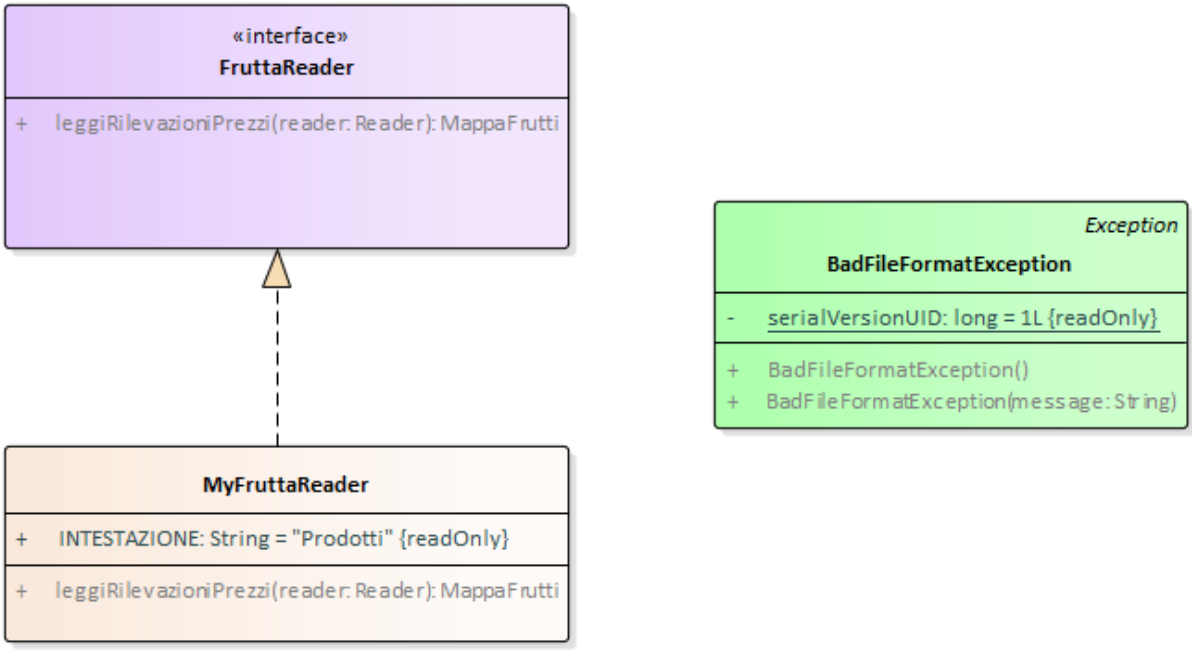
- l'identificativo univoco della frutta, nella forma "`nomefrutta - nomesottotipo`" (NON è garantita la presenza di spazi intorno al separatore "'lineetta')
- i singoli prezzi, nel formato con simbolo di valuta che precede l'importo **se il prezzo è stato rilevato in quella data, o la stringa "NaN" nel caso in cui il prezzo non sia stato rilevato in quella data** (frutto fuori stagione): si tenga presente che la stringa "NaN" potrebbe essere preceduta o seguita da spazi.

```
Prodotti;10/02/25;17/02/25;24/02/25;03/03/25;...;03/05/25;04/05/25
Albicocche - nostrane;NaN;NaN;NaN;NaN;... ;€ 2,21;€ 2,00
Ciliegie - Bigarreau; NaN;NaN;NaN;NaN;...;€ 5,50;€ 4,63
...
Pere - Abate fetel;€ 1,85;€ 1,81;€ 1,81;€ 1,81;...;NaN;NaN
...
```

SEMANTICA:

a) L'interfaccia ***FruttaReader*** (fornita) dichiara il metodo *leggiRilevazioniPrezzi,* che legge dal ***Reader*** fornito (già aperto) i dati e restituisce l'insieme delle rilevazioni lette, sotto forma di ***MappaFrutti***; lancia ***BadFileFormatException*** con opportuno messaggio d'errore in caso di problemi nel formato del file, o ***IOException*** in caso di altri problemi di I/O.

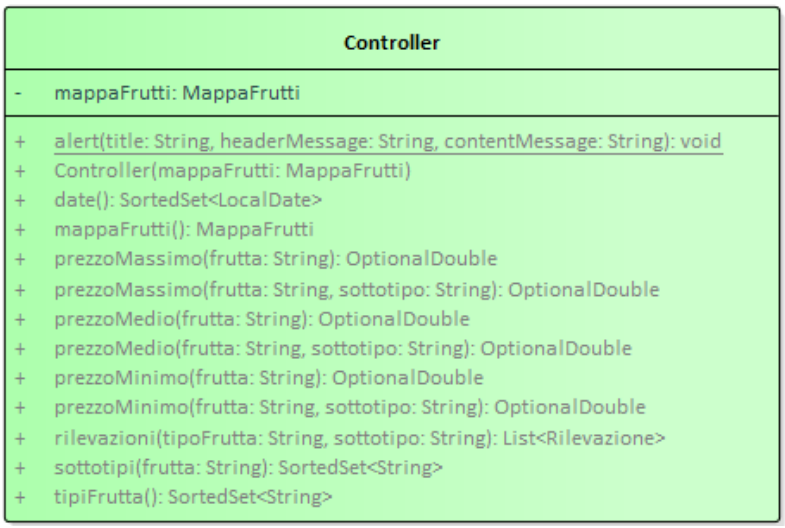
b) La classe ***MyFruttaReader*** (da realizzare) implementa ***FruttaReader***:
   - non c'è alcun costruttore
   - Il metodo *leggiRilevazioniPrezzi* (da implementare) effettua le letture: lancia ***NullPointerException*** in caso di reader nullo, o ***BadFileFormatException*** con dettagliato messaggio d'errore in caso di mancato rispetto del formato previsto, catturando eventuali altre eccezioni interne. In particolare deve verificare:
     - per quanto riguarda la riga di intestazione:
       - che abbia come primo elemento la parole chiave prevista
       - che le date successive, *di cui occorre memorizzare il numero ai fini delle verifiche successive,* siano tutte nel corretto formato italiano SHORT
     - per quanto riguarda le righe successive:
       - che siano presenti *tanti prezzi* (o "NaN") *quante le date* specificate nella riga di intestazione
       - che inizino con l'identificativo univoco della specifica frutta, nel formato previsto.



Parte 3 (punti: 10)

Package: frutta.controller *(punti 0)*

In questa applicazione, il controller costituisce un mero ponte fra model e view: crea la struttura dati e le inoltra le richieste provenienti dalla view. A tal fine è organizzato come segue:



SEMANTICA:

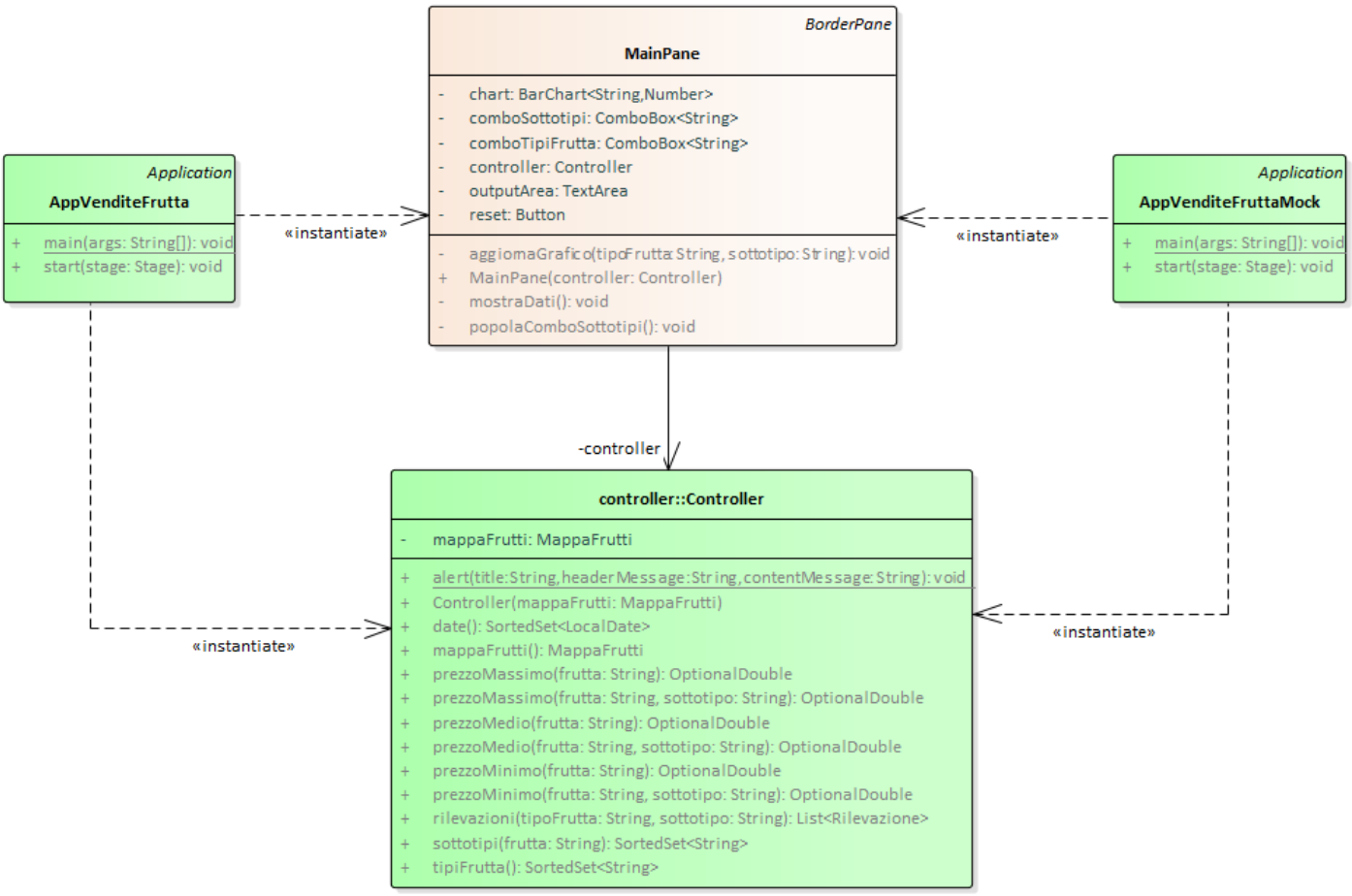
a) La classe ***Controller*** (fornita) riceve e gestisce al suo interno il ***MappaFrutti*** a cui delega praticamente tutte le operazioni, di quasi identica denominazione.

   Il metodo statico *alert*, utilizzabile dal ***MainPane*** quando è attiva la grafica, consente di far comparire all'utente una finestra di dialogo che mostri il messaggio d'errore specificato.

Package: frutta.ui **[TEMPO STIMATO: 45-60 minuti]** *(punti 10)*

La classe ***AppVenditeFrutta*** (fornita) costituisce l'applicazione JavaFX che si occupa di aprire i file, creare il controller e incorporare il ***MainPane***. Per consentire di collaudare la GUI anche in assenza / in caso di malfunzionamento della parte di persistenza, è possibile avviare l'applicazione mediante la classe ***AppVenditeFruttaMock***.

L'interfaccia utente è illustrata nelle figure seguenti e segue il modello sotto illustrato*:*



- <u>In alto</u> sono presenti le due combo per la scelta del tipo e del sottotipo di frutta.
- <u>Al centro</u> vi è l'area di testo per i messaggi.
- <u>In basso</u> vi è l'area per il grafico e il pulsante RESET.

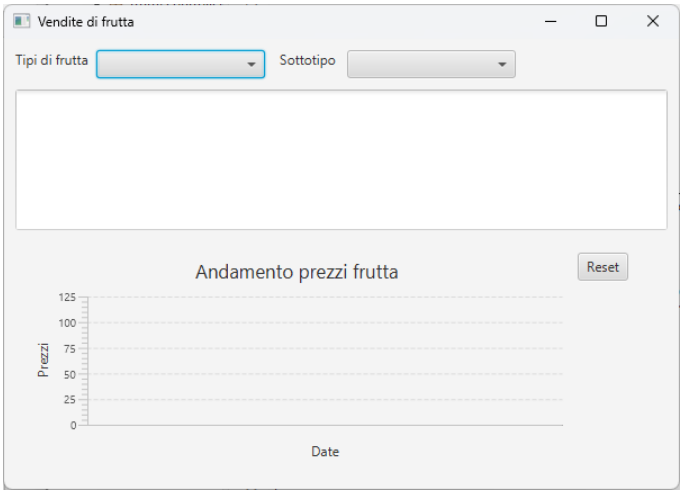


Fig. 1: situazione iniziale.

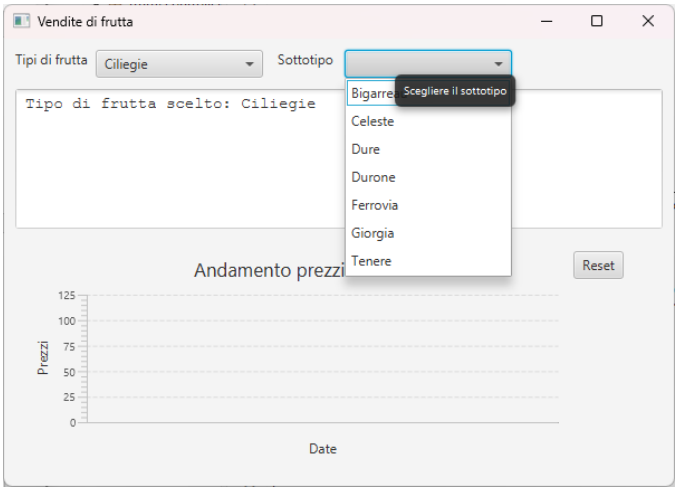


Fig. 2 situazione dopo la scelta del tipo di frutta

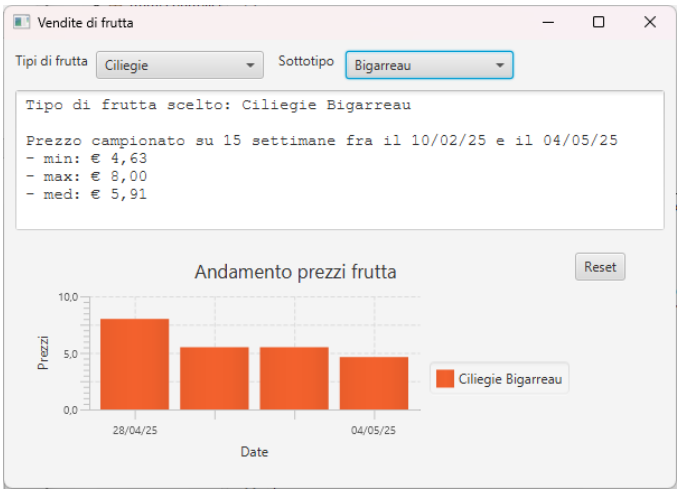


Fig. 3: situazione dopo la scelta del sottotipo.

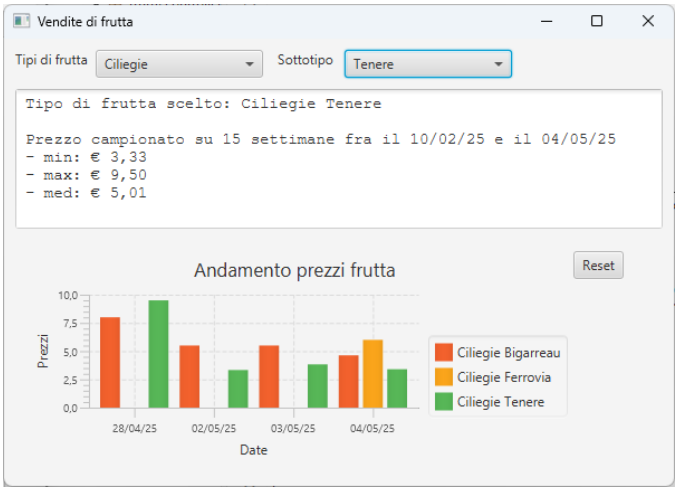


Fig. 4 evoluzione scegliendo altri sottotipi

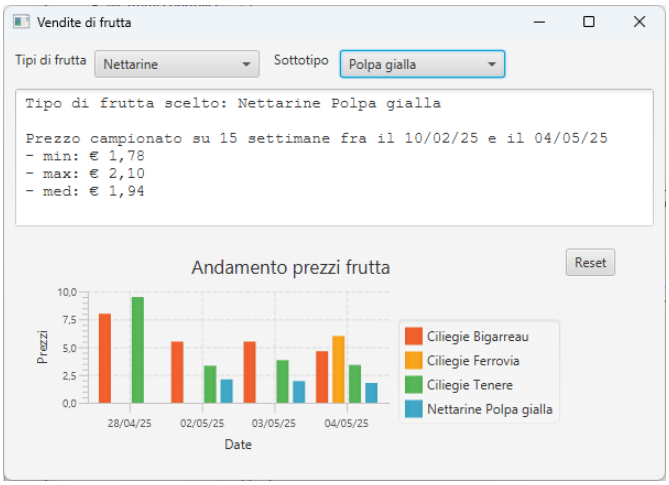


Fig. 5: prenotazione con esito POSITIVO.

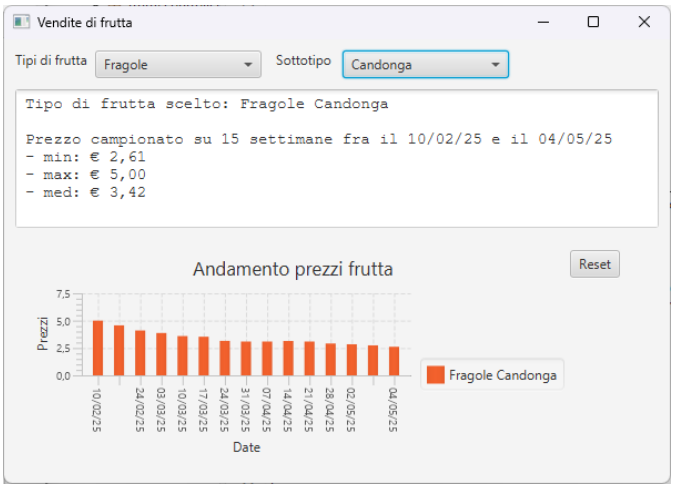


Fig. 6 prenotazione con esito NEGATIVO

Inizialmente è popolata la sola combo dei tipi di frutta (Fig.1). Scegliendone uno, viene popolata di conseguenza la combo a fianco, con i relativi sottotipi (Fig. 2). Scegliendo anche quest'ultimo (Fig.3), la relativa serie di prezzi viene graficata e nell'area di testo vengono mostrati i prezzi minimo, medio e massimo, specificando su quante settimane è fatto il campionamento, con precisazione della data iniziale e finale. **È richiesto di emettere le stringhe mostrate in figura.**

Continuando a scegliere ulteriori sottotipi (Fig. 4) o anche nuovi tipi/sottotipi di frutta (Fig. 5), i relativi dati vengono aggiunti al grafico e alla relativa legenda; l'area di testo mostra invece sempre e solo i dati dell'ultima serie inserita.

Il pulsante RESET consente in qualunque momento di azzerare il grafico, così da ripartire con un grafico pulito aggiungendo nuove serie successive (Fig. 6).

Il MainPane è fornito *quasi completamente realizzato*: è presente tutta la parte strutturale, mentre rimane da realizzare il gestore degli eventi *mostraDati*, insieme al suo metodo ausiliario *aggiornaGrafico*.

In particolare:

- *popolaComboSottotipi* (fornito) viene invocato all'atto della selezione della combo dei tipi di frutta: nel caso non sia selezionato nulla, emette un messaggio d'errore tramite il metodo *alert* del ***Controller***. Se tutto procede normalmente, emette sull'area di testo il tipo di frutta scelto e procede al popolamento della combo sottotipi richiedendo al controller i dati corrispondenti al tipo di frutta selezionato.
- *mostraDati* **(da realizzare)** è associata alla selezione della combo sottotipi e scatena il vero calcolo:

- se i valori di tipo/sottotipo sono nulli, il metodo deve ritornare senza fare nulla;
- altrimenti, deve recuperare tipo e sottotipo di frutta, e da quelli, interrogando il ***Controller,*** recuperare i dati sui prezzi e sintetizzare il messaggio da emettere a video; poi, tramite il metodo ausiliario *aggiornaGrafico*, provvedere all'aggiornamento del grafico
- l'aggiornamento del grafico, implementato nel metodo *aggiornaGrafico*, deve consistere nel:
  - verificare preliminarmente se lo specifico tipo e sottotipo di frutta siano già presenti nel grafico (SUGGERIMENTO: si frutti l'apposito metodo *serieGiàPresente*): in tal caso non devono essere ri-aggiunti, quindi l'aggiornamento termina qui.
  - altrimenti, recuperare la lista delle rilevazioni per lo specifico tipo e sottotipo di frutta
  - creare la nuova serie dati, darle il giusto nome e popolarla aggiungendo, per ogni rilevazione, la coppia (data formattata, prezzo)
  - aggiungere al grafico la nuova serie così creata e rendere visibile la legenda (se già non lo era)

Cose da ricordare

- salva costantemente il tuo lavoro: l'informatica a volte può essere "subdolamente ostile"…
- in particolare: se ora compila e stai per fare modifiche, salva la versione attuale (non si sa mai)

Checklist di consegna

- Hai fatto un **JAR eseguibile**, che contenga cioè l'indicazione del main?
- Hai controllato che **si compili** e **ci sia tutto**? [NB: non includere il PDF del testo]
- Hai **rinominato** IL PROGETTO, lo ZIP e il JAR esattamente come richiesto?
- Hai **chiamato** la cartella del progetto esattamente come richiesto?
- Hai fatto un **unico file ZIP (NON .7z, rar o altri formati)** contenente l'intero progetto? In particolare, ti sei assicurato di aver incluso tutti i file .java (e non solo i .class)?
- **Hai consegnato DUE file distinti, ossia lo ZIP col progetto e il JAR eseguibile?**
- Su EOL, hai **premuto** il tasto "CONFERMA" per inviare il tuo elaborato?